Manual de instalación: Entorno DevOps

Metodologías y Procesos Software

Jaime Marrero Torrejón
Juan Neva Vargas
Ana Isabel Brenes Ramírez
Kevin Bauti Huelva

Universidad de Cádiz

Escuela superior de ingeniería

Grado de ingeniería informática



Índice

Índice **2**
1. Introducción y requisitos previos **3**
2. Instalación de herramienta base **3**
2.1 Docker Engine 3
2.2 SDK de .NET (C#) y xUnit 4
2.3 Java (JDK) y Gradle 4
3. Preparación del Proyecto y Dependencias **5**
4. Despliegue del servidor de CI/CD (GoCD) **6**
4.1 Archivo docker-compose.yml 6
4.2 Iniciar GoCD 6
5. Verificación del Ciclo de Vida DevOps **7**

1. Introducción y requisitos previos

Este manual de instalación está diseñado para levantar el entorno DevOps creado por nuestro grupo. El entorno está preparado para ejecutarse sobre **Windows con WSL2 (Ubuntu)** o directamente sobre una distribución **Linux (Ubuntu/Debian)**.

- **Sistema Operativo:** Windows 10/11 con WSL2 (Ubuntu 22.04+) o Linux nativo.
- **Hardware recomendado:** Mínimo 8 GB de RAM y virtualización activada en BIOS.
- **Editor recomendado:** Visual Studio Code.
- **Github**: Control de versiones y repositorio remoto del código.
- **Gradle**: Orquestador de tareas de construcción y pruebas.
- **xUnit**: Framework para las pruebas unitarias en C# **(.NET 8)**.
- **Docker**: Empaquetado de la aplicación y de los servicios en contenedores.
- **GoCD**: Servidor de automatización (CI/CD) que ejecuta la *pipeline* completa.

2. Instalación de herramienta base

2.1 Docker Engine

Docker se encarga de aislar la ejecución de GoCD y de empaquetar la aplicación en contenedores reproducibles.

Ejecutar en la terminal de Linux/WSL:

```shell
# 1. Limpieza de versiones previas
sudo apt update && sudo apt remove -y docker docker-engine docker.io containerd runc

# 2. Dependencias del sistema
sudo apt install -y ca-certificates curl gnupg lsb-release

# 3. Repositorio oficial de Docker
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
```

```
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update

# 4. Instalación de Docker y Docker Compose
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# 5. Configuración de permisos sin sudo
sudo usermod -aG docker $USER
newgrp docker

# 6. Arranque del servicio
sudo service docker start
```

Verificación:

```
Shell
docker run hello-world
```

2.2 SDK de .NET (C#) y xUnit

Proporciona el compilador del lenguaje, las librerías base y el motor de ejecución de pruebas unitarias.

```
Shell
sudo apt update
sudo apt install -y dotnet-sdk-8.0
```

Verificación:

```
Shell
dotnet --version
# Debe devolver: 8.0.x
```

2.3 Java (JDK) y Gradle

Gradle actúa como el orquestador general de tareas del proyecto (compilación, ejecución de xUnit y generación de binarios).

```shell
# 1. Instalar Java (requerido por Gradle)
sudo apt update
sudo apt install -y default-jdk unzip wget

# 2. Descargar e instalar Gradle oficial
export GRADLE_VERSION=8.10.2
wget https://services.gradle.org/distributions/gradle-${GRADLE_VERSION}-bin.zip -P /tmp
sudo mkdir -p /opt/gradle
sudo unzip -d /opt/gradle /tmp/gradle-${GRADLE_VERSION}-bin.zip
sudo ln -sf /opt/gradle/gradle-${GRADLE_VERSION}/bin/gradle /usr/bin/gradle
```

Verificación:

```shell
java -version
gradle -v
```

3. Preparación del Proyecto y Dependencias

Clonar el repositorio desde GitHub y situarse en la raíz del proyecto:

```shell
git clone https://github.com/<tu-organizacion-o-usuario>/<tu-repositorio>.git
cd <tu-repositorio>
```

Para asegurar que Gradle y .NET se comuniquen correctamente y las pruebas de xUnit se ejecuten en local, correr el wrapper de Gradle:

```shell
./gradlew test
```

El resultado debe finalizar con el estado **`BUILD SUCCESSFUL`**.

4. Despliegue del servidor de CI/CD (GoCD)

GoCD se levanta de forma automatizada mediante contenedores para evitar configuraciones complejas en la máquina anfitriona.

4.1 Archivo docker-compose.yml

En la raíz del proyecto se encuentra el archivo de orquestación de GoCD:

```none
services:
  gocd-server:
    image: gocd/gocd-server:v23.5.0
    container_name: gocd-server
    ports:
      - "8153:8153"
    volumes:
      - gocd-server-data:/godata
    restart: unless-stopped

  gocd-agent:
    build:
      context: .
      dockerfile: Dockerfile.agent
    container_name: gocd-agent
    environment:
      - GO_SERVER_URL=http://gocd-server:8153/go
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    depends_on:
      - gocd-server
    restart: unless-stopped

volumes:
  gocd-server-data:
```

Nota sobre el Agente: El agente utiliza un `Dockerfile.agent` personalizado que incluye Java 17, el SDK de .NET 8 y el cliente de Docker para que pueda compilar el código de C# y

construir las imágenes de forma autónoma.

4.2 Iniciar GoCD

```shell
docker compose up -d
```

Una vez levantados los contenedores, acceder al panel de control desde el navegador:

- http://localhost:8153

5. Verificación del Ciclo de Vida DevOps

Para comprobar que el entorno está operativo de extremo a extremo:

1. **Revisión del Agente:** En la interfaz de GoCD (`http://localhost:8153`), entrar a la pestaña **Agents** y verificar que el agente aparezca en estado `Idle` (listo para recibir tareas).
2. **Ejecución de Pipeline:** Realizar un commit y `git push` a GitHub.
3. **Comprobación de Etapas:**
    - **Stage 1 (Build & Test):** GoCD invoca `./gradlew test` y valida las pruebas unitarias de xUnit.
    - **Stage 2 (Package & Docker):** Si las pruebas están en verde, se genera la imagen Docker.
4. **Demostración de Cambio:** Al agregar una funcionalidad y subir el código, GoCD debe dispararse de nuevo en automático, pasar los nuevos tests y generar la nueva versión del contenedor sin intervención manual.